

Data cleaning

This was a relatively large dataset that strictly contains categorical data. Some of the columns such as `precinct_id` and `neighborhood_id` initially appeared to be numerical, but really those were numerical codes that have no meaning in terms of magnitude.

It was necessary to clean some nonsense from the original dataset. There were occasional cells that were just two digit numbers found in columns like `TU1_VEHICLE_TYPE`. We replaced data like that with `None`.

The hardest decision was what to do with blank columns. This data is all from police reports, so depending on what actually happened in the accident, many columns would be left blank. For example, there were two sets of columns that would report on secondary and tertiary collisions, like when one vehicle rear ends a second vehicle, then the second vehicle hits a 3rd vehicle. There were many reports that were just single vehicle accidents where many columns were left blank. The most sparse rows came from hit and runs, likely from when someone walked out to their car parked on the street and found it damaged.

Before One-hot Encoding

The columns before our one-hot encoding were named as follows:

1. 'offense_code'
2. 'district_id'
3. 'precinct_id'
4. 'neighborhood_id'
5. 'bicycle_ind'
6. 'pedestrian_ind'
7. 'HARMFUL_EVENT_SEQ_1'
8. 'HARMFUL_EVENT_SEQ_2'
9. 'HARMFUL_EVENT_SEQ_3'
10. 'road_location'
11. 'ROAD_DESCRIPTION'
12. 'ROAD_CONTOUR'
13. 'ROAD_CONDITION'
14. 'LIGHT_CONDITION'
15. 'TU1_VEHICLE_TYPE'
16. 'TU1_TRAVEL_DIRECTION'
17. 'TU1_VEHICLE_MOVEMENT'
18. 'TU1_DRIVER_ACTION'
19. 'TU1_DRIVER_HUMANCONTRIBFACTOR'
20. 'TU1_PEDESTRIAN_ACTION'
21. 'TU2_VEHICLE_TYPE'
22. 'TU2_TRAVEL_DIRECTION'
23. 'TU2_VEHICLE_MOVEMENT'
24. 'TU2_DRIVER_ACTION'
25. 'TU2_DRIVER_HUMANCONTRIBFACTOR'
26. 'TU2_PEDESTRIAN_ACTION'
27. 'hour'
28. 'day_of_week'
29. 'month'

These columns can be explored in `eda.ipynb`. The `.value_counts()` for each category are present here.

The target: `high_risk`

For our target `y` column, we decided to combine two columns called `SERIOUSLY_INJURED` or `FATALITIES`. This would indicate the seriousness of the accident to indicate if people were hurt or if lives were lost.

After One-Hot Encoding

After one-hot encoding the data exploded into a massive 976 columns.

Class imbalance

A big obstacle with this dataset is the class imbalance. We find that there are 275685 NOT high risk accidents with only 6559 rows that ARE high risk.

Dimensionality reduction (or not)

Because the one-hot encoded data has 976 features it would seem that reducing the dimensions of that dataset would be necessary. Because it was all one-hot encoded, PCA was not an option. We could have potentially done some called Single Value Decomposition (which was not covered in class). But after some research, it seemed that the Decision Tree based models we were planning on using may actually have their performance hindered by dimensionality reduction. So we made the decision to forego reduction and let the models do their thing.

The search for a model

There were many types of models explored. We expected the best performance from models based on Decision Trees due to the categorical data. We did a grid search with the following models:

Tree based:

- RandomForrest

- ExtraTrees
- LightGBM (a model developed by Microsoft)

These had OK performance with F1 scores of about 35%.

Linear Classifiers:

- LogisticRegression
- LinearSVC

We found that the only way to get any kind of results from the linear based classifiers was by heavily weighting the rows with positive `high_risk`. That then lead to models that were heavily skewed towards recall (hovering about 90% scores) and poor precision (with 10% precision scores).

Test/Train Split, Cross-Validation, and Grid Search

An 80/20 test/train split was applied to the data, and the test set was preserved until it was needed after the models had been trained. The training set was further split into 3 cross validation folds before being used to perform a grid search to find the best hyperparameters for our models. We used 3 folds instead of 5 because some of the grid searches were taking an extremely long time. This ensures that the best hyperparameters would be found for a generalized fit. After identifying the optimal hyperparameters, the models were re-trained on the entire training set in hopes that it would improve the results. This was done because the positive class is sparse, we wanted to provide as many positive training examples as possible.

Feature importance

By looking at the `.feature_importances_` of the tree-based models and the `.coef_` values of the linear classifier models, we can discern which categories were most consequential in contributing to the predictions. For all models, if there was a motorcycle involved or if the driver was charged with "Reckless Driving" score high within the top 20 features. See `evaluate.ipynb` for in-depth results.

The "common sense" knowledge that motorcycles are much more dangerous than cars indicates that these models are learning correlations that apply to real life. Another highly predictive category was if a DUI/DWAI was involved.

We were able to use the `shap.TreeExplainer` library to evaluate some of the categories that the LightGBM model. One of the columns that stands out was the `hour` of the day, and it showed that the hours from 10pm to 2am were most dangerous.

Ensemble

Since we didn't have any one good model, we decided to make an ensemble. Using the `.predict_proba()` to get per-row 'probabilities' allowed us to create a soft-voting ensemble. The ensemble's results are as follows:

```
--- Results for Weighted Soft Voting Ensemble (Threshold: 0.55) ---
      precision    recall  f1-score   support

     0       0.9880     0.9722     0.9800     55137
     1       0.3006     0.5023     0.3761     1312

 accuracy          0.9613     56449
 macro avg       0.6443     0.7372     0.6781     56449
weighted avg       0.9720     0.9613     0.9660     56449

Confusion Matrix:
[[53604  1533]
 [   653    659]]
```

As we can see the recall was about 50% for our positive class, meaning of the actual rows that were present as `high_risk`, we caught about half of them. The precision score was almost exactly 30%, meaning that of all the times the modeled guessed it was `high_risk`, it was correct about 30% of the time.

Interpretation

The final results may not seem impressive, but we believe this type of data is just not very consistent to begin with. In reality, it is very difficult to predict when an automobile accident is going to be dangerous or not. There are all types of vehicles with a wide variety of safety ratings that are hard to predict. We believe our models would have been more successful with more detailed data, such as the actual vehicle models or if there was a column that indicated speed. Unfortunately the best we had to go on was the `Motorcycle` category and whether the officer on scene decided to charge them with "Reckless Driving" or a "DUI/DWAI". Also, accidents that happened from the hours of 10pm to 2am were much more high risk than other times of the day.